

Swinburne University of Technology
COS30049 - Computing Technology Innovation Project
Semester January, 2026
Tutorial Class: Wednesday, 2:00pm - 6:00pm
Tutor: Ly Quynh Tran and Dr. Dang Phuoc Nhat

Assignment 2
AI-based Project for Cybersecurity (AI4Cyber)

Team TechNova
Nguyen Thanh Kien - 105507742
Bui Tien Hung - 105555411
Truong Nam Hung - 105556375
Nguyen Huu Hieu - 104988566

Submission Date: 15th March, 2026

Introduction.....	3
Problem Framing.....	3
Data Collection.....	3
Data Processing.....	4
Data Analysis.....	6
Machine Learning Model Selection.....	7
Naive Bayes.....	8
Support Vector Machine (LinearSVC).....	8
Logistic Regression.....	8
Random Forest.....	8
K-Means Clustering.....	9
Model Comparison and Final Selection.....	9
Technical Implementation.....	9
Implementation Evaluation.....	10
Model Performance Metrics	11
Cross Validation	11
Error Analysis and Classifier Reliability	11
Spam Campaign Clustering	13
Conclusion.....	14
Bibliography.....	15
Appendix	15

Introduction

The rapid growth of messaging platforms such as Telegram has created a new vector for spam and phishing attacks targeting everyday users. Unlike email spam, messages received in personal chat carry an implicit sense of trust, making users more vulnerable to deception. In this project, we develop a Spam Detection Tool that applies Natural Language Processing and supervised machine learning to classify Telegram messages as spam or legitimate, an approach demonstrated as the most effective method for detecting spam in messaging platforms (Ahmadi, Wang and Zhao 2025).

Problem Framing

Our main objective is to develop a machine learning solution capable of accurately classifying Telegram messages as spam or ham, helping users avoid phishing and deceptive content. Traditional approaches such as keyword blacklists and rule-based filters fail to adapt as attackers evolve their tactics, so a machine learning approach is more appropriate as it learns patterns from labelled data and generalises to unseen spam variants. Our specific goals are:

1. Establish a baseline using a Naive Bayes classifier trained on TF-IDF vectorised text data.
2. Evaluate whether SVM improves on the baseline by identifying a more optimal decision boundary, particularly in reducing false positives.
3. Compare all models using accuracy, precision, recall, and F1 score to determine the most effective solution for deployment.

Data Collection

We sourced two datasets from Kaggle. The primary dataset, *spam_Emails_data*, contains 215,910 records across two columns: Text and Label, with a near-equal class distribution of 52.5% ham and 47.5% spam. The secondary dataset, *Spam-50k*, contains over 50,000 rows across five columns: Message ID, Subject, Message, Spam/Ham label, and Date. Both originate from email communication records, and their linguistic structure is directly transferable to messaging spam detection, making them suitable for training a Telegram-focused classifier.

We evaluated candidate datasets based on three criteria: size, class balance, and relevance to messaging spam patterns. After merging both sources, we dropped the Message ID column as it carries no predictive value, then applied deduplication on the combined text field to remove repeated records. This left a final corpus of 194,575 unique records with a balanced 52.5% ham and 47.5% spam distribution, reducing the risk of model bias toward either class.

Message ID	Subject	Message	Spam/Ham	Date
0	christmas tree farm pictures		ham	12/10/1999
1	vastar resources, inc.	-----	ham	12/13/1999
2	calpine daily gas nomination	- calpine daily gas	ham	12/14/1999
3	re : issue	/ 99 .	ham	12/14/1999
4	meter 7268 nov allocation	thanks .	ham	12/14/1999
5	mc mullen gas for 11 / 99	thanks	ham	12/14/1999
6	meter 1517 - jan 1999	hlc	ham	12/14/1999
7	duns number changes	3 - 7578	ham	12/14/1999
8	king ranch	9643 .	ham	12/14/1999
9	re : entex transition	distributions list	ham	12/14/1999
10	entex transition	distributions list	ham	12/14/1999
11	1st rev dec. 1999 josey ranch nom	713 - 964 - 9434	ham	12/15/1999
12	2nd rev dec. 1999 josey ranch nom	713 - 964 - 9434	ham	12/15/1999
13	unify close schedule	melissa x 35615	ham	12/15/1999
14	meter 1431 - nov 1999	hlc	ham	12/15/1999
15	meter 1431 - nov 1999	hlc	ham	12/15/1999
16	y 2 k - texas log	990 - 8332	ham	12/15/1999
17	re : lyondell citgo	hlc	ham	12/16/1999
18	hpl fuel gas buy - back for december 1999	thanks	ham	12/16/1999
19	ua 4 - meter 1440 for 11 / 97 - fal furris	thanks , stella	ham	12/16/1999
20	ua 4 for meter 8808 - 6 / 98 - deal 96731	thanks , stella	ham	12/16/1999
21	january spot tickets	x 3 - 6353	ham	12/17/1999
22	pennenergy property details	- devon stx . xls	ham	12/17/1999
23	miscellaneous	david	ham	12/17/1999
24	re : purge of old contract _event _status	d . n .	ham	12/20/1999
25	out on vacation	romeo d 'souza	ham	12/20/1999
26	re : meter 1428 oct 99 - beaumont methanol	3 - 7159	ham	12/20/1999
27	re : evergreen deals	d	ham	12/20/1999

Figure 1: Spam-50k dataset file

Data Processing

Raw data from open-source platforms requires significant cleaning before it can be used for machine learning. The Spam-50k dataset in particular contained inconsistent labelling and unstructured text that needed preparation before feeding into any classifier.

We began by dropping unnamed index columns and standardising labels to lowercase, retaining only valid "spam" or "ham" rows. We then merged the Subject and Message columns into a single combined text field, since spam signals appear across both and treating them separately would result in incomplete feature representation. Missing values were filled with empty strings before merging, and duplicates were removed based on the combined field.

Text preprocessing then converted raw language into a clean, machine-readable format. Each message was lowercased, and URLs, email addresses, and numeric values were replaced with standardised placeholders to reduce vocabulary noise while preserving structural signals. We retained characters such as "!", "\$", and "?" as they appear disproportionately in spam and carry discriminative value, while removing other punctuation. Tokenization was applied using NLTK (Bird and Loper 2004), followed by stopword removal. The cleaned tokens were then transformed into numerical features using TF-IDF vectorisation (Mallampati and Hegde 2022) with 10,000 maximum features, a unigram and bigram range, minimum document frequency of 5, and maximum document frequency of 0.8 to filter both rare and overly common terms. We

used a stratified 70/15/15 train/validation/test split so hyperparameter tuning could be performed on the validation set without contaminating the final test evaluation.

Beyond text, we engineered structural metadata features to capture behavioural patterns that linguistic features alone may not reveal. These included message length, word count, and special character count. Temporal features - hour of day and whether the message was sent on a weekend - were simulated to reflect real-world spam sending patterns. All numerical features were scaled using MinMaxScaler and combined with TF-IDF features through a ColumnTransformer, creating a hybrid feature set providing both linguistic and structural signals.

Data Analysis

We conducted exploratory data analysis on the cleaned dataset to uncover patterns and inform modelling decisions.

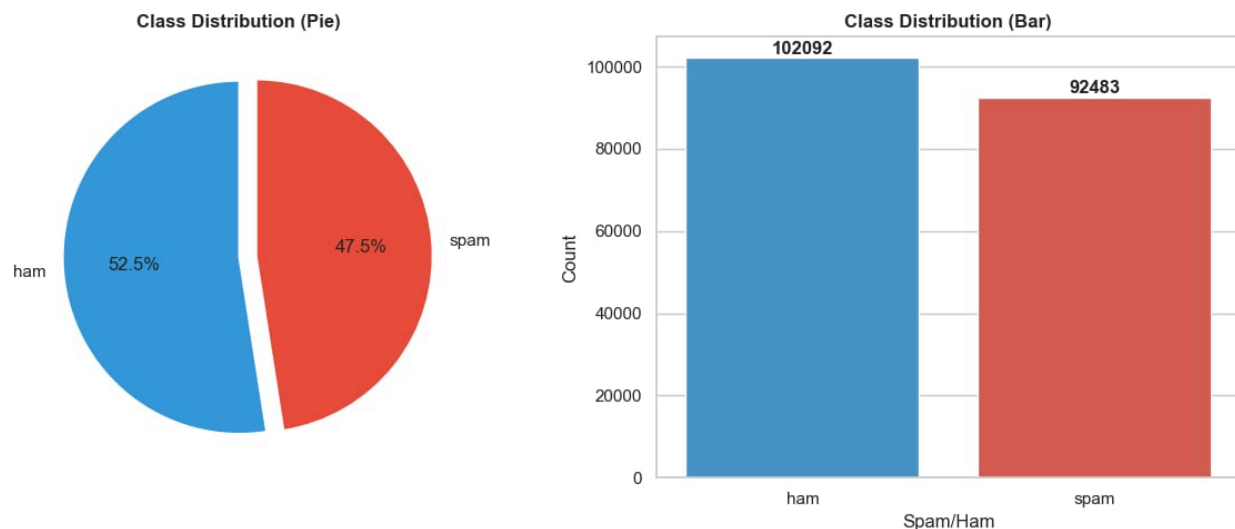


Figure 2: Class distribution of Spam/Ham messages

Figure 2 illustrates the class distribution across the combined 194,575 records, revealing a near-equal split of 52.5% ham and 47.5% spam. This balance is significant as it ensures evaluation metrics such as precision and recall reflect genuine model performance rather than class frequency.

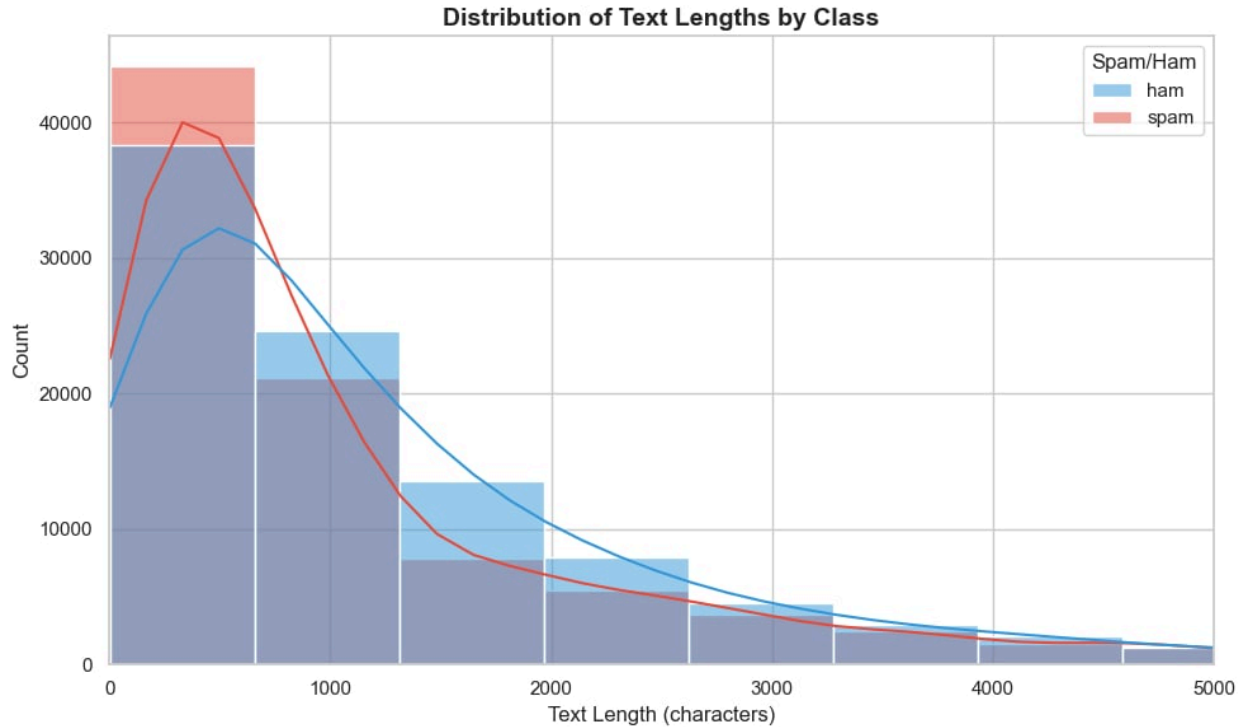


Figure 3: Distribution of Text Lengths by Class

Figure 3 presents the distribution of text lengths by class, revealing a meaningful structural difference. Ham messages follow a gradual declining distribution with a long tail beyond 5,000 characters, while spam messages are concentrated in shorter ranges with a sharp peak between 200 and 400 characters, dropping to near zero beyond 1,000. This confirmed that spam tends to be more concise and formulaic, which directly motivated the inclusion of text length and word count as metadata features alongside TF-IDF in the hybrid pipeline.

Machine Learning Model Selection

Models were selected based on the problem characteristics and suitability for high-dimensional sparse text features produced by TF-IDF.

Naive Bayes

Naive Bayes was selected as the baseline due to its well-established effectiveness with text data. Its probabilistic assumption that each word contributes independently aligns naturally with TF-IDF term frequencies, and its low computational cost makes it an appropriate benchmarking starting point (Oyeyemi and Ojo 2023). We considered Decision Tree as an alternative but found it prone to overfitting on sparse text features.

Support Vector Machine (LinearSVC)

SVM (LinearSVC) was selected as a primary model due to its strong performance in large vocabulary feature spaces. LinearSVC identifies an optimal margin boundary between spam and ham, making it more robust to noise than probabilistic approaches (Oyeyemi and Ojo 2023). As it does not natively support probability estimation, we wrapped it in a CalibratedClassifierCV to enable confidence score outputs required for ROC curve evaluation. Kernel SVM was considered but rejected due to significantly higher computational cost without meaningful accuracy gains on text classification.

Logistic Regression

Logistic Regression was included for its interpretability and reliability in binary classification. Unlike Naive Bayes, it models class membership probability directly without assuming feature independence, and provides probability estimates useful for threshold tuning in deployment. We considered Ridge Classifier as an alternative but found Logistic Regression more suitable due to its native probability outputs.

Random Forest

Random Forest was selected as an ensemble method to evaluate whether non-linear patterns could be captured that linear models may miss. All four classifiers were evaluated within both the text-only and metadata pipelines, though Random Forest was of particular interest in the metadata setting as its tree-based structure naturally handles mixed feature types. We considered XGBoost as an alternative but found it more complex to integrate into the existing sklearn Pipeline without meaningful performance gains.

K-Means Clustering

K-Means Clustering was included as an additional unsupervised method to discover hidden patterns within the spam class. Rather than classifying messages, K-Means groups spam records by linguistic similarity to reveal campaign types. We set $k=4$ based on domain inspection, identifying four distinct campaign types: phishing, software piracy, pharmacy spam, and B2B marketing. DBSCAN was considered but K-Means was selected for its computational efficiency on high-dimensional sparse feature spaces (Sinaga and Yang 2020).

Metadata pipeline

Metadata Pipeline was developed as a hybrid approach using ColumnTransformer to combine TF-IDF text features with numerical structural features. This was motivated by our EDA finding that spam exhibits consistent behavioural patterns beyond linguistic content, including shorter message lengths and higher special character density. All four classifiers were trained and evaluated within this pipeline to assess whether structural features produced measurable gains over text features alone.

Model Comparison and Final Selection

Our team assessed all four classification models based on their theoretical suitability for the problem before conducting empirical evaluation. We determined that while Naive Bayes and Logistic Regression provide strong and interpretable linear baselines, SVM and Random Forest are better positioned to generalise across diverse spam patterns due to their stronger handling of high-dimensional feature spaces and non-linear relationships respectively. K-Means was retained as a complementary unsupervised method rather than a replacement for the classifiers, as its purpose is pattern discovery within the spam class rather than binary prediction. The metadata pipeline was developed as a complementary hybrid approach, combining structural and linguistic signals within the same classifier framework to evaluate whether behavioural features produce measurable gains over text features alone. The final selection of models reflects our goal of covering a range of algorithmic approaches to ensure a thorough and comparative evaluation.

Technical Implementation

The implementation was carried out entirely in Python. Data handling relied on Pandas and NumPy, NLTK provided tokenisation and stopwords removal, and Scikit-learn supplied the TF-IDF vectoriser, all classifiers, Pipeline construction, and GridSearchCV (Pedregosa et al. 2011). Visualisation was handled through Matplotlib and Seaborn, and trained models were exported using Joblib.

A key decision was wrapping each model and its TF-IDF vectoriser into a unified Scikit-learn Pipeline object, ensuring preprocessing and classification steps were applied consistently across training and testing phases and preventing data leakage. Two additional implementations were developed: the metadata pipeline described above, and an unsupervised K-Means clustering pipeline applied exclusively to spam records with PCA projection for visualisation. A data augmentation step was also implemented using nlpaug to generate synthetic variants through three strategies - synonym replacement, random word deletion, and character swapping - to stress-test model robustness (Feng et al. 2021).

During development, we encountered two notable challenges. First, raw CSV files contained non-standard encodings that caused the default UTF-8 parser to fail, so we implemented a fallback to latin-1 encoding. Second, standard GridSearchCV cross-validation posed a data leakage risk between training and validation sets, so we used PredefinedSplit to ensure the validation set remained strictly separate throughout tuning, identifying the optimal regularisation parameter C for LinearSVC across values of 0.1, 1.0, and 10.0. The nlpaug augmentation process was computationally expensive, so we limited it to a representative sample of 200 records.

Implementation Evaluation

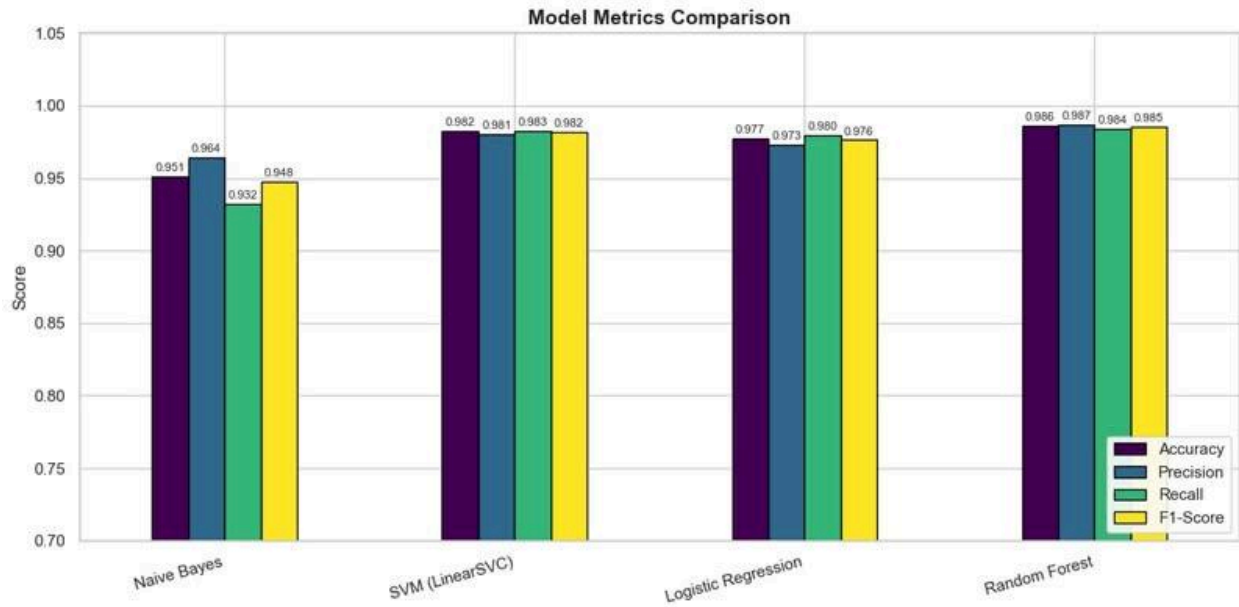


Figure 4: Model Metrics Comparison

Model Performance: Figure 4 presents metrics for all four classifiers. Random Forest achieved the highest overall performance with accuracy 0.986, precision 0.987, recall 0.984, and F1 0.985. SVM delivered strong results with accuracy 0.982 and F1 0.982, while Logistic Regression performed reliably at F1 0.976. Naive Bayes, though achieving accuracy 0.951, showed notably lower recall of 0.932, indicating it missed more spam messages than the other models.

Cross Validation: We conducted 5-fold stratified cross validation to confirm consistency across data subsets. All four models showed stable performance with low standard error. SVM maintained the highest mean F1 of 0.9866 across all folds, reinforcing its selection as the recommended deployment model.

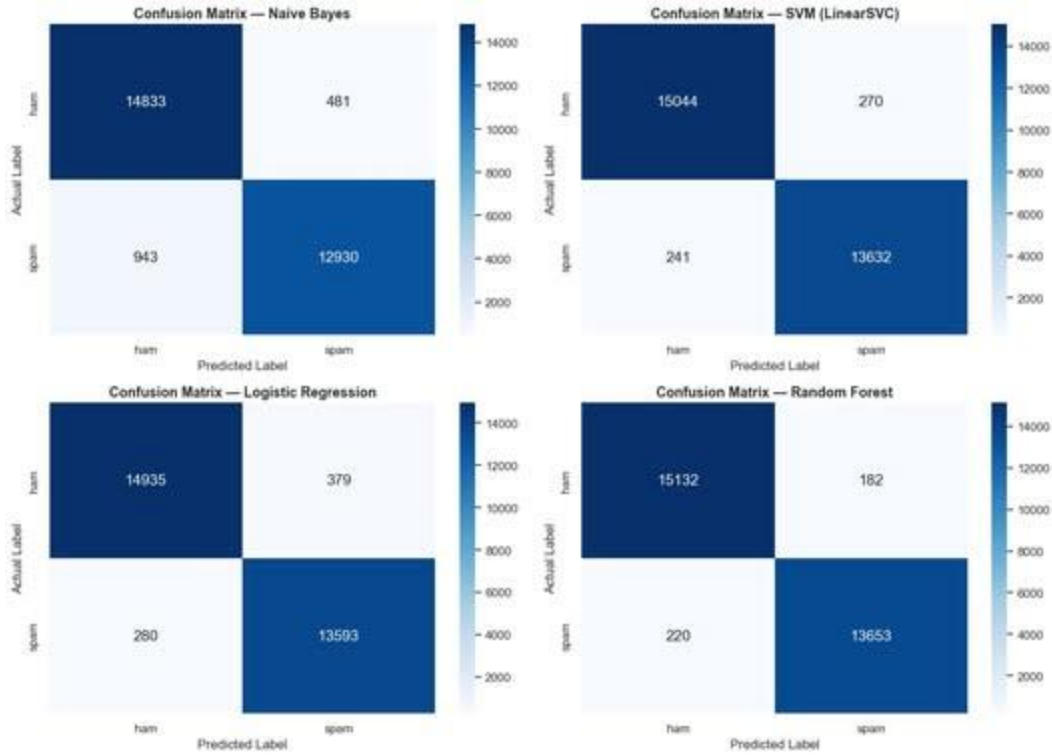


Figure 5: Confusion Matrices

Figure 5 shows confusion matrices for all models. Random Forest produced the fewest false positives (182) compared to SVM (270), Logistic Regression (379), and Naive Bayes (481). However, Random Forest generated 220 false negatives versus SVM's 241, a marginal difference. Figure 6 presents ROC curves, where Random Forest achieved the highest AUC of 0.999, followed by SVM at 0.998, Logistic Regression at 0.997, and Naive Bayes at 0.992. While Random Forest marginally outperforms SVM on aggregate metrics, SVM's more conservative false positive balance makes it the better choice where incorrectly blocking legitimate messages carries a significant usability cost.

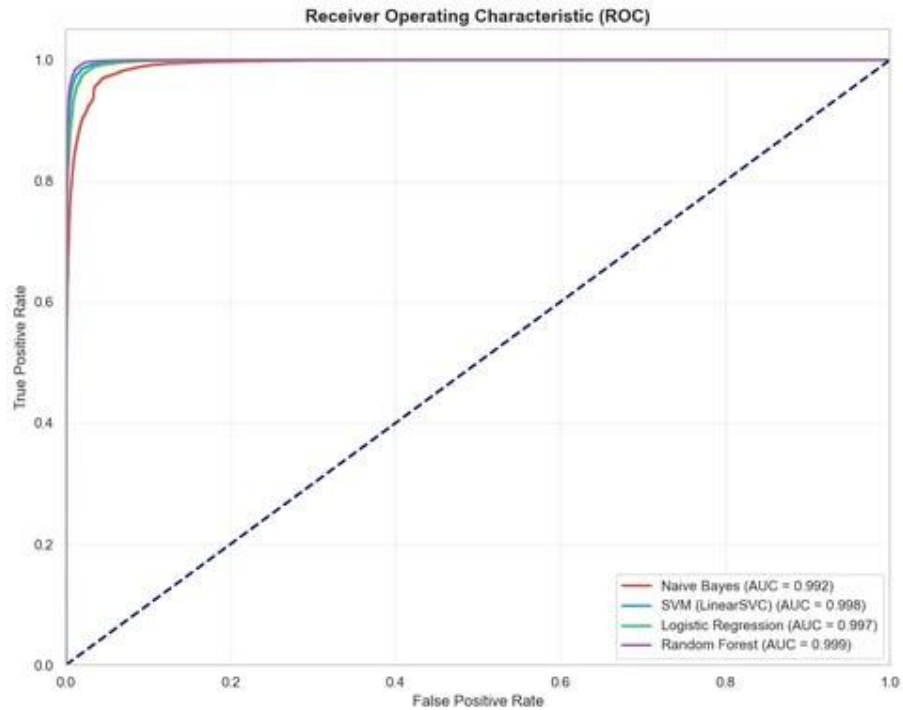


Figure 6: Receiver Operating Characteristic (ROC) Curves

The near perfect AUC scores across all models indicate strong overall discriminative ability, however our team determined that SVM represents the best overall balance between minimising false positives and maintaining high recall. While Random Forest marginally outperforms SVM on aggregate metrics, the difference is negligible at this scale, and SVM's slightly lower false positive count relative to its false negative count makes it marginally more conservative in incorrectly blocking legitimate messages, which carries a significant usability cost in real-world deployment.

Spam Campaign Clustering

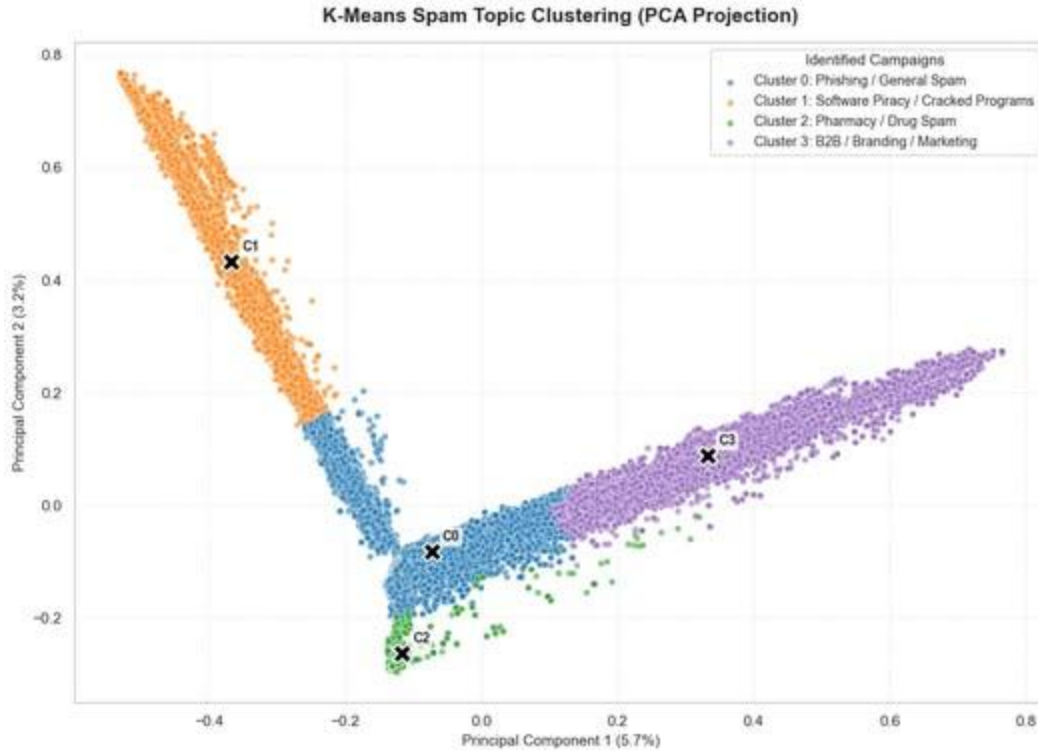


Figure 7: K-Means Spam Topic Clustering (PCA Projection)

Figure 7 presents the K-Means PCA projection, with the two retained components explaining 5.7% and 3.2% of total variance. The well-separated cluster regions indicate that the identified campaign types are linguistically distinct. Cluster 0 represents phishing and general spam, Cluster 1 software piracy, Cluster 2 pharmacy spam, and Cluster 3 B2B marketing content. This analysis revealed that spam is not homogeneous but organised into distinct thematic campaigns, providing actionable intelligence beyond binary classification.

Conclusion

This project successfully developed a machine learning solution for detecting spam in Telegram-focused contexts, addressing all three goals defined in Problem Framing. While Random Forest achieved the highest aggregate metrics, SVM (LinearSVC) was selected as the recommended deployment model due to its superior cross-validation stability with a mean F1 of 0.9866 and its more conservative balance between false positives and false negatives. The K-Means clustering analysis further revealed four distinct spam campaign types, providing deeper insight beyond binary classification.

Several limitations remain. Temporal features were simulated rather than derived from real timestamps, and both datasets originate from email records rather than native Telegram

messages. Future work would benefit from platform-specific data collection and exploration of transformer-based models to address increasingly sophisticated spam tactics.

Bibliography

- Ahmadi, M., Wang, L. and Zhao, Y. 2025, 'Leveraging Large Language Models for Cybersecurity: Enhancing SMS Spam Detection with Robust and Context-Aware Text Classification', *arXiv*, viewed 15 March 2026, <<https://arxiv.org/abs/2502.11014>>.
- Bird, S. and Loper, E. 2004, 'NLTK: The Natural Language Toolkit', *Proceedings of the ACL Interactive Poster and Demonstration Sessions*, Association for Computational Linguistics, Barcelona, pp. 214–217, viewed 15 March 2026, <<https://aclanthology.org/P04-3031/>>.
- Feng, S., Gangal, V., Wei, J., Chandar, S., Vosoughi, S., Mitamura, T. and Hovy, E. 2021, 'A Survey of Data Augmentation Approaches for NLP', *Findings of the Association for Computational Linguistics*, arXiv, viewed 15 March 2026, <<https://arxiv.org/abs/2105.03075>>.
- Mallampati, D. and Hegde, N.P. 2022, 'Feature Extraction and Classification of Email Spam Detection Using IMTF-IDF and Skip-Thought Vectors', *Ingenierie des Systemes d'Information*, vol. 27, no. 6, pp. 951–958, viewed 15 March 2026, <<https://iieta.org/journals/isi/paper/10.18280/isi.270610>>.
- Oyeyemi, A. and Ojo, S. 2023, 'SMS Spam Detection and Classification to Combat Abuse Using Machine Learning', *Journal of Advances in Mathematics and Computer Science*, vol. 38, no. 10, pp. 144–156, viewed 15 March 2026, <<https://arxiv.org/pdf/2406.06578>>.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, E. 2011, 'Scikit-learn: Machine Learning in Python', *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, viewed 15 March 2026, <<https://jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf>>.
- Sinaga, K.P. and Yang, M.S. 2020, 'Unsupervised K-Means Clustering Algorithm', *IEEE Access*, vol. 8, pp. 80716–80727, viewed 15 March 2026, <<https://doi.org/10.1109/ACCESS.2020.2988796>>.